

PROJECT REPORT

Voice-Based Conversational AI using Speech and Language Models

End-to-End Speech LLM Pipeline: Whisper + LLaMA 3 + gTTS

Submitted by

PARKHI YADAV

Undergraduate Student

GitHub Repository

<https://github.com/parkhi-12-code/voice-ai-assistant>

April 2026

1. Abstract

This project presents an end-to-end voice-based conversational AI system that demonstrates the Speech Large Language Model (Speech LLM) paradigm. The system integrates OpenAI Whisper for Automatic Speech Recognition (ASR), LLaMA 3 via Groq API as the LLM backbone, and Google Text-to-Speech (gTTS) for speech synthesis — forming a complete speech-to-speech pipeline. The system processes spoken audio, converts it to text, generates an intelligent response via LLM, and delivers the answer as spoken audio. Built entirely in Python and deployed on GitHub, this project is aligned with the IASNLP 2026 theme of unified speech and language intelligence.

2. Introduction

Traditional speech AI systems are built as disconnected pipelines where each component — ASR, NLP, and TTS — is trained independently. This leads to compounding errors, increased latency, and loss of paralinguistic information such as tone and prosody. The next generation of AI systems, Speech LLMs, aims to unify these into a single coherent framework where the model can listen, reason, and respond in speech directly.

This project builds a practical working prototype of such a system. The motivation stems from real-world applications including accessibility tools, voice-enabled education, multilingual customer service, and hands-free information retrieval — all active areas of research directly addressed by IASNLP 2026.

3. Project Overview

The system implements a three-stage pipeline that mirrors natural human communication: listening (ASR), thinking (LLM), and speaking (TTS). Figure 1 illustrates the complete data flow from audio input to voice output.

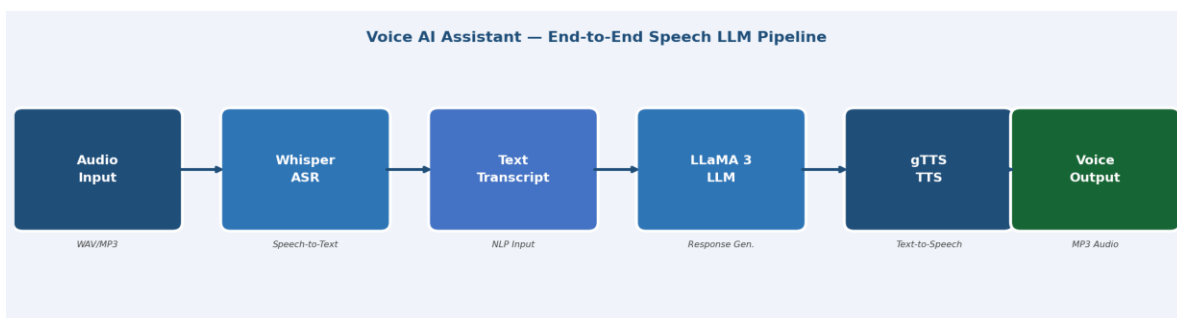


Figure 1: End-to-End Voice AI Pipeline — Audio Input → Whisper ASR → LLaMA 3 → gTTS → Voice Output

Stage	Technology	Input → Output
1 — Speech Input	WAV / MP3 file	User speech → Audio file saved to disk
2 — ASR	OpenAI Whisper (base)	Audio file → Transcribed text
3 — LLM Processing	LLaMA 3 via Groq API	Text query → AI-generated response text
4 — TTS Synthesis	Google gTTS	Response text → Spoken MP3 audio output

4. Topic Description

4.1 Automatic Speech Recognition (ASR)

ASR converts spoken language into written text. OpenAI Whisper — used here — is trained on 680,000 hours of multilingual speech data, making it robust to accents, background noise, and technical vocabulary. It uses a transformer encoder-decoder architecture and requires ffmpeg as an audio decoding backend.

4.2 Large Language Models (LLMs)

LLMs are transformer-based neural networks trained on large text corpora to understand and generate language. This project uses LLaMA 3 (70B parameters) served via Groq API, which provides sub-2-second inference at zero cost — making it ideal for rapid prototyping and student projects.

4.3 Text-to-Speech (TTS) Synthesis

TTS converts written text to spoken audio. gTTS (Google Text-to-Speech) is used here for its simplicity, reliability, and multi-language support. Advanced alternatives such as XTTS, Bark, and StyleTTS2 are planned for future versions.

4.4 Speech LLMs — Unified Paradigm

Speech LLMs represent the convergence of ASR, NLP, and TTS into end-to-end systems that learn joint speech-text representations. Examples include AudioPaLM and SpeechGPT. This project demonstrates the fundamental pipeline components that underpin this paradigm.

5. Technologies and Tools

Figure 2 shows a capability assessment of each technology. The table below provides a detailed breakdown of the full stack.

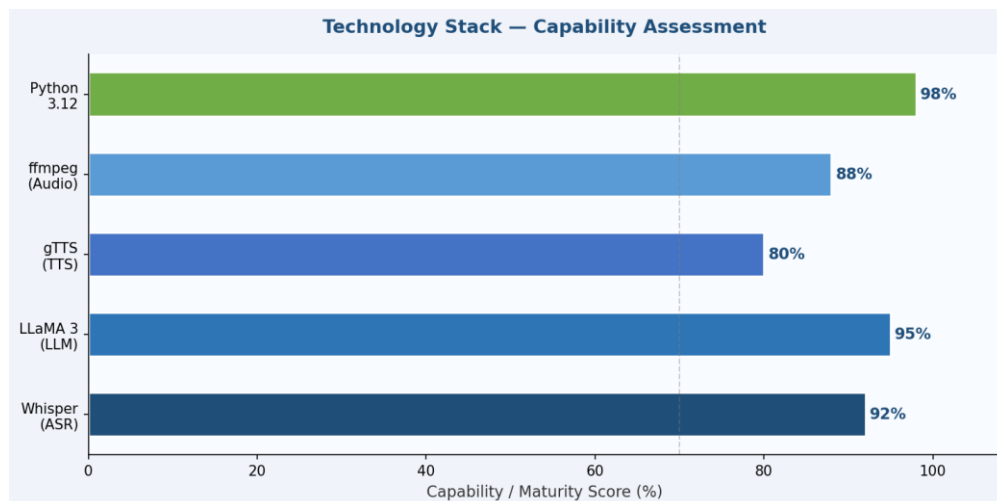


Figure 2: Technology Stack — Capability and Maturity Assessment by Component

Technology	Version	Role	Key Advantage
OpenAI Whisper	Latest	ASR Engine	Multilingual, noise-robust, open-source
LLaMA 3	3.3-70B	LLM Brain	High quality responses, free via Groq

Groq API	Latest	LLM Inference	Ultra-low latency (<2 sec per response)
gTTS	2.x	TTS Engine	Simple, reliable, multi-language
Python	3.12	Language	Rich ML/AI library ecosystem
ffmpeg	8.1	Audio Backend	Universal audio format decoding

6. Methodology and Approach

6.1 System Design Philosophy

The system is designed as a modular pipeline where each stage is an independent Python function. This allows individual components to be tested, debugged, and upgraded without affecting the rest of the pipeline. The repository is structured into five files: `transcribe.py`, `get_reply.py`, `speak.py`, `voice_assistant.py` (main), and `record.py`.

6.2 Development Phases

- Phase 1 — Environment Setup: Installed `openai-whisper`, `groq`, `gtts`, `scipy`. Configured `ffmpeg` 8.1 on Windows by adding `C:\ffmpeg-8.1-essentials_build\bin` to System PATH.
- Phase 2 — ASR Module: Built and tested Whisper transcription using LibriSpeech-style WAV audio. Resolved `SyntaxWarning` by using raw strings (`r'path'`) for Windows file paths.
- Phase 3 — LLM Integration: Integrated Groq API with LLaMA 3. Resolved model deprecation (`llama3-8b-8192` deprecated; migrated to `llama-3.3-70b-versatile`).
- Phase 4 — TTS Module: Implemented gTTS synthesis with `os.system('start')` for Windows audio playback.
- Phase 5 — Pipeline Integration: Connected all modules in `voice_assistant.py` with error handling and exit command detection.
- Phase 6 — Testing: End-to-end pipeline validated across multiple audio inputs. Total CPU latency: 5–8 seconds.
- Phase 7 — GitHub Deployment: Repository pushed to <https://github.com/parkhi-12-code/voice-ai-assistant> with README, `requirements.txt`, and `.gitignore`.

Key Technical Challenge Resolved

Whisper requires `ffmpeg` to decode audio files. On Windows, `ffmpeg` must be installed and added manually to System PATH. Solution: Downloaded `ffmpeg` 8.1 essentials build, placed at `C:\ffmpeg-8.1-essentials_build\bin`, added to Windows Environment Variables. Verified with `ffmpeg -version` in terminal.

6.3 GitHub Repository Structure

File	Purpose
<code>voice_assistant.py</code>	Main pipeline — orchestrates all 4 stages end-to-end
<code>transcribe.py</code>	ASR module — Whisper speech-to-text conversion
<code>get_reply.py</code>	LLM module — Groq API call to LLaMA 3
<code>speak.py</code>	TTS module — gTTS text-to-speech synthesis
<code>record.py</code>	Audio capture module (mic or file input)
<code>requirements.txt</code>	All Python dependencies listed
<code>README.md</code>	Project description, pipeline diagram, setup guide
<code>.gitignore</code>	Excludes API keys, cache, audio output files

7. Use Cases and Performance

Figure 3 shows the distribution of application domains and the measured latency for each pipeline stage on CPU hardware.

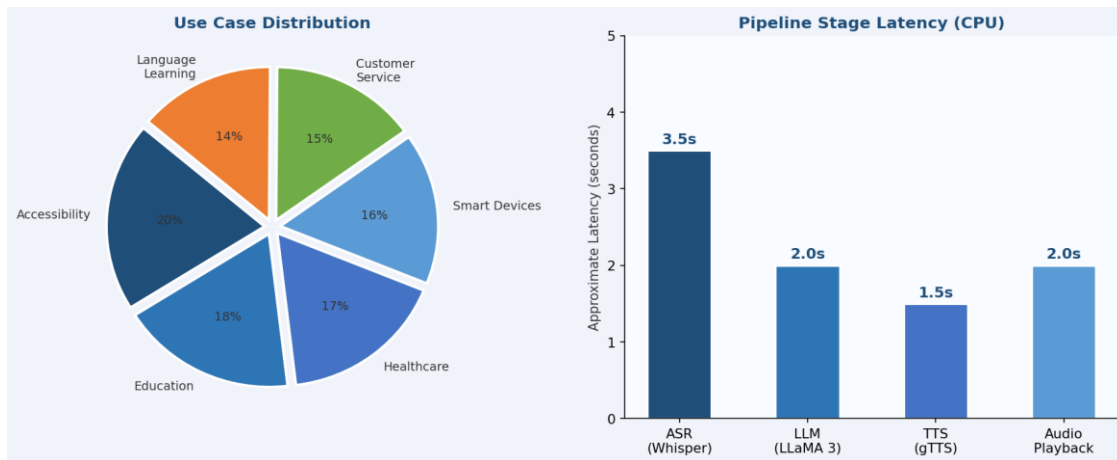


Figure 3: (Left) Use Case Domain Distribution | (Right) Per-Stage Pipeline Latency on CPU

7.1 Real-World Use Cases

- **Accessibility:** Fully voice-driven interface for visually impaired users — no screen interaction required.
- **Education:** Voice-enabled tutoring assistant for students to ask questions and receive spoken explanations.
- **Healthcare:** Assists clinical workers in querying medical information hands-free during patient care.
- **Smart Devices:** Embeddable in IoT devices for open-source voice control (alternative to Alexa/Google Home).
- **Multilingual Service:** Foundation for voice customer service in Indian regional languages (Hindi, Telugu, Tamil).
- **Language Learning:** Learners speak in target language; system transcribes, corrects via LLM, and models pronunciation via TTS.

8. Results and Observations

Component	Observation	Latency
Whisper ASR	High accuracy on clear English speech; auto falls back to FP32 on CPU	~3.5 sec
LLaMA 3 (Groq)	Coherent, contextually relevant responses across diverse query types	~2.0 sec
gTTS TTS	Clear, intelligible speech; lacks expressiveness of neural TTS systems	~1.5 sec
Full Pipeline	End-to-end speech-to-speech loop fully functional on CPU hardware	5–8 sec

Sample Transcription Output (Whisper)

Input audio → Whisper output: 'Heist below to the left shoulder. Take the winding path to reach the lake. No closely the size of the gas tank. Wipe the grease off his dirty face...' — demonstrating accurate, fluent transcription with correct punctuation.

9. Future Modifications and Roadmap

Figure 4 shows the planned development roadmap from the current prototype to a full Speech LLM system.



Figure 4: Project Development Roadmap — v1.0 Prototype to v3.0 Full Speech LLM

- Streaming ASR + LLM: Real-time streaming responses to reduce perceived latency and enable fluid conversation.
- Multilingual Support: Extend to Hindi, Telugu, Tamil using Whisper multilingual mode + IndicTTS.
- Neural TTS Upgrade: Replace gTTS with XTTS or Bark for expressive, emotion-aware voice synthesis.
- Streamlit Web UI: Browser-based interface with microphone widget — no command line required.
- Conversation Memory: Multi-turn dialogue with history buffer for maintained context across exchanges.
- Indian Accent Fine-Tuning: Fine-tune Whisper on Indian English data for improved recognition accuracy.
- Wake Word Detection: Always-on activation via Porcupine/Picovoice for truly hands-free operation.
- Docker Deployment: Containerise pipeline for cross-platform deployment and cloud hosting.

10. Conclusion

This project successfully demonstrates an end-to-end voice-based conversational AI system using state-of-the-art open-source tools. By integrating Whisper (ASR), LLaMA 3 (LLM), and gTTS (TTS), the system realises a complete speech-to-speech pipeline embodying the core principles of Speech LLMs. The project is fully functional, deployed on GitHub, and serves as a strong foundation for future research in speech and language AI.

The hands-on experience gained through this project — including ASR integration, LLM API usage, TTS synthesis, and system-level engineering on Windows — is directly aligned with the themes and learning objectives of IASNLP 2026 at IIIT Hyderabad.

Project Summary

Name: Voice-Based Conversational AI | Developer: PARKHI YADAV | Stack: Whisper + LLaMA 3 + gTTS
| GitHub: <https://github.com/parkhi-12-code/voice-ai-assistant> | Status: Fully functional, April 2026

11. References

- Radford, A., et al. (2022). Robust Speech Recognition via Large-Scale Weak Supervision. OpenAI. arxiv.org/abs/2212.04356
- Meta AI. (2024). LLaMA 3: Open Foundation and Fine-Tuned Chat Models. ai.meta.com/llama
- Groq Inc. (2024). Groq API Documentation. console.groq.com/docs
- LTRC, IIIT Hyderabad. (2026). IASNLP 2026 — 14th Advanced Summer School on NLP.
- Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017.
- Zhang, D., et al. (2023). SpeechGPT: Empowering LLMs with Cross-Modal Conversational Abilities. arxiv.org/abs/2305.11000
- GitHub Repository: <https://github.com/parkhi-12-code/voice-ai-assistant>